RISC-V LLVM 后端源码阅读教程

> 从三个小案例出发，沿着 LLVM IR 到 RISC-V 汇编和目标文件的真实源码路径阅读。

源码基准：LLVM `main` 提交 `043acd707` 本地源码根目录：`/root/MIC-for-learning/llvm-project` 适合读者：熟悉基本 C/C++、LLVM IR 和简单 RISC-V 汇编，准备第一次系统阅读 LLVM 后端。

使用说明

这份教程的目标不是逐行讲完 `llvm/lib/Target/RISCV`，而是让你能回答三个具体问题：

1. `add i64` 为什么会变成 RISC-V `add`?
2. 参数、返回值和栈帧由哪些源码共同实现?
3. RVV 指令为什么需要 `vsetvli`，LLVM 又在哪里插入它?

你不需要先完整编译当前 LLVM。阅读源码时用 `rg` 定位稳定的文件名、类名和函数名；行号只作辅助，因为上游源码会不断变化。本机若有 `llc`，可以运行实验，但先检查版本：

```
llc --version
git -C /root/MIC-for-learning/llvm-project rev-parse --short HEAD
```

系统中的 `llc` 可能不是提交 `043acd707` 构建出来的。出现输出差异时，先判断是版本差异、优化级别差异，还是目标特性差异。

建议每个案例都按同一节奏阅读：先看测试的输入和期望输出，再画数据形态变化，随后进入实现，最后修改一个变量重新验证。不要从一个大文件第一行顺序读到最后一行。

目录

1. 先建立后端地图
2. 案例一：整数加法如何变成 add
3. 案例二：函数调用与栈帧
4. 案例三：RVV 与 vsetvli
5. 形成自己的源码阅读方法

先建立后端地图

一条主线，五种主要数据形态

先把后端理解成一系列“逐步增加目标信息”的转换：

```
C/C++
  |
  | Clang 前端
  v
LLVM IR                          与具体寄存器和编码无关
  |
  | 合法化、SelectionDAG 指令选择
  v
MachineInstr                     已是目标机器指令或 pseudo
  |
  | 寄存器分配、调度、栈帧处理、pseudo 展开
  v
最终 MachineInstr
  |
  | AsmPrinter / MC lowering
  v
MCInst                           面向汇编器和编码器的轻量指令
  |                  |
  | MCInstPrinter    | MCCodeEmitter + object writer
  v                  v
汇编文本             目标文件中的字节、重定位和节
```

LLVM IR 仍使用无限数量的 SSA 值。SelectionDAG 是本教程主线中的指令选择表示，它会经历类型和操作合法化。MachineInstr 已经能表达 RISC-V 的 ADD、PseudoCALL、PseudoVADD... 等操作，但早期仍可使用虚拟寄存器和 pseudo。MCInst 不再承担复杂的代码生成分析，主要服务于汇编输出和机器码编码。

> 阅读提示：不要把“选中了 RISC-V 指令”和“写出了最终机器码”看作同一步。寄存器分配、栈帧处理、pseudo 展开和 MC 编码都发生在这两点之间。

后端关键部件各管什么

部件	主要问题	当前源码入口
`RISCVTargetMachine`	目标实例怎样创建，怎样按函数取得 subtarget	`RISCVTargetMachine.cpp`
`RISCVSubtarget`	当前 CPU、扩展和调度模型支持什么	`RISCVSubtarget.cpp/.h`
CodeGen pass pipeline	指令选择和各个机器级 pass 以什么顺序运行	`RISCVCodeGenPassBuilder.cpp`、`RISCVTargetMachine.cpp`
`RISCVTargetLowering`	IR 类型和操作怎样合法化、调用怎样 lowering	`RISCVISelLowering.cpp/.h`
TableGen 描述	指令格式、编码、寄存器类、pattern 怎样声明	`RISCVInstrInfo.td` 及被包含的 `.td` 文件
DAG selector	自动生成 pattern 未覆盖的选择怎样手写处理	`RISCVISelDAGToDAG.cpp`
MachineFunction passes	栈帧、frame index、RVV 配置等怎样完成	`RISCVFrameLowering.cpp`、`RISCVRegisterInfo.cpp`、`RISCVInsertVSETVLI.cpp`
MC 层	`MachineInstr` 怎样变成文本或编码字节	`RISCVAsmPrinter.cpp`、`MCTargetDesc/RISCVMCCodeEmitter.cpp`

在当前源码中，`RISCVCodeGenPassBuilder::addInstSelector` 加入 `RISCVISelDAGToDAGPass`。这给出本教程采用 SelectionDAG 主线的直接证据。LLVM 同时含有 GlobalISel 基础设施和 RISC-V 相关实现，但它不是这里追踪普通测试时采用的主线。以后遇到 `IRTranslator`、`Legalizer`、`RegBankSelect`、`InstructionSelect` 等名字时，再把它们归入 GlobalISel 路径。

TargetMachine、Subtarget 和函数属性

`RISCVTargetMachine` 表示一个编译目标。`getSubtargetImpl(const Function &F)` 会读取函数的 `target-cpu`、`tune-cpu` 和 `target-features` 属性，再取得对应的 `RISCVSubtarget`。因此同一个模块中的不同函数可以拥有不同目标特性。

`RISCVSubtarget` 保存由 TableGen 生成和手写代码共同建立的特性查询，例如 `hasVInstructions()`。阅读某条指令为什么可用时，常见路径是：

```
命令行 -mattr / 函数 target-features
        -> RISCVTargetMachine::getSubtargetImpl
        -> RISCVSubtarget::ParseSubtargetFeatures
        -> hasStdExt... / hasVInstructions 等查询
        -> TableGen Predicate 或 C++ 条件
```

ISA、ABI、`-march` 和 `-mabi`

ISA 决定处理器能执行哪些指令和扩展；ABI 决定独立编译单元怎样互相调用与组织数据。你的概括“ISA 决定指令集扩展，ABI 决定调用约定”是正确主线，还可补上数据模型和对象文件约定。

选项	主要影响	典型例子
`-march` / target features	可选择的指令、寄存器和扩展能力	`rv64gc`、`+v`、`+zba`
`-mabi`	参数和返回值位置、寄存器保存责任、类型宽度等接口规则	`lp64`、`lp64d`、`ilp32`

两者必须相容。例如 ABI 要求硬浮点参数寄存器时，目标必须具备相应浮点能力。后端里的 `Subtarget` 更靠近 ISA/微架构能力；`RISCVCallingConv.cpp`、`RISCVISelLowering.cpp` 和 frame lowering 则共同落实 ABI。

TableGen 应该怎样读

TableGen 是描述语言，也是代码生成工具。`.td` 文件里的 `def`、`class`、`multiclass` 和 `Pat` 会生成 C++ 表，不是在运行时解释。阅读一条指令时分三层看：

1. 指令格式和编码字段，例如 `RVInstR`。
2. 具体指令记录，例如 `def ADD : ALU_rr<...>`。
3. 从 DAG 操作到指令的匹配，例如 `def : PatGprGpr<add, ADD>`。

不要只在 `RISCVISelDAGToDAG.cpp` 中搜索 `ADD`。普通 pattern 大量来自 TableGen 自动生成的 selector，手写 `Select` 主要补充复杂情况。

建立自己的定位命令

以下命令都从 LLVM 源码根目录运行：

```
cd /root/MIC-for-learning/llvm-project
rg -n 'addInstSelector' llvm/lib/Target/RISCV
rg -n '^def ADD|PatGprGpr<add' llvm/lib/Target/RISCV/RISCVInstrInfo.td
rg -n 'emitInstruction' llvm/lib/Target/RISCV/RISCVAsmPrinter.cpp
rg -n 'encodeInstruction' llvm/lib/Target/RISCV/MCTargetDesc/RISCVMCCodeEmitter.cpp
```

完成本章后，你应该能解释每一种表示比上一种增加了哪些 RISC-V 信息，并能指出“指令选择”和“指令编码”对应的不同入口。

案例一：整数加法如何变成 add

从 CodeGen 测试开始

入口是 `llvm/test/CodeGen/RISCV/alu64.ll`。其中的 `@add` 同时检查 RV64 和 RV32：

```
define i64 @add(i64 %a, i64 %b) nounwind {
  %1 = add i64 %a, %b
  ret i64 %1
}
```

RV64 期望的核心结果很短：

```
add a0, a0, a1
ret
```

同一个 `i64 add` 在 RV32 上却需要多条指令，因为 XLEN 只有 32 位。测试本身已经告诉我们：IR 操作名相同，并不保证一对一映射到机器指令；类型合法性和目标宽度先决定表示方法。

测试顶部的 `RUN` 行是可复现规范：`llc -mtriple=riscv64 -verify-machineinstrs` 生成结果，再由 `FileCheck` 检查。源码阅读时先回答四个问题：输入类型是什么、triple 是什么、启用了哪些 feature、检查的是精确序列还是局部性质。

若本机 `llc` 可用，可以把最小函数保存到 `/tmp/add.ll`：

```
llc -mtriple=riscv64 -O2 -o - /tmp/add.ll
llc -mtriple=riscv32 -O2 -o - /tmp/add.ll
```

第一步：合法化和 TargetLowering

进入 `RISCVISelLowering.cpp` 时，不要期待一定存在名为 `LowerADD` 的函数。`RISCVTargetLowering` 的构造过程会声明合法类型、寄存器类和各类操作的 action；通用合法化器据此决定一个节点可以保留、提升、展开或自定义 lowering。

对本例，RV64 的 `i64` 与 XLen 一致，DAG 中的普通 `ISD::ADD` 能直接进入 pattern 匹配。RV32 的 `i64` 不是单个 GPR 的原生整数宽度，合法化后会形成低 32 位、高 32 位和进位处理。这正是测试中多条 `add`、`sltu` 的来源。

观察问题：

- `i64` 在 RV64 与 RV32 上分别对应什么寄存器表示？
- `add i64` 是否走了 target-specific custom lowering？
- `-O0` 和 `-O2` 是否改变合法化结论，还是主要改变周边优化？

第二步：在 TableGen 中找到选择规则

`RISCVInstrInfo.td` 中有两处需要连起来：

```
def ADD : ALU_rr<0b0000000, 0b000, "add", Commutable=1>;
def : PatGprGpr<add, ADD>;
```

第一条定义 RISC-V `ADD` 的操作数形状、汇编名和编码字段继承关系；第二条把 SelectionDAG 的 `add` 节点匹配到 `ADD`。同一段还有：

```
def : PatGprSimm12<add, ADDI>;
```

所以 `%a + 1` 能选择 `addi`，而两个寄存器相加选择 `add`。这里要区分三个名字：LLVM IR 文本中的 `add`、SelectionDAG 的 `ISD::ADD`、RISC-V 机器操作码 `ADD`。

`ALU_rr` 继续继承 `RVInstR`。沿继承链可看到输出 `GPR:$rd`、输入 `GPR:$rs1` 和 `GPR:$rs2`，以及 R 型指令的 `funct7`、`funct3`、opcode 字段。调度资源 `Sched<[WriteIALU, ReadIALU, ReadIALU]>` 描述调度模型，不负责定义 ISA 语义。

第三步：匹配虚拟机与手写 Select 的边界

`RISCVDAGToDAGISel::Select(SDNode *Node)` 位于 `RISCVISelDAGToDAG.cpp`。函数先处理已经是 machine opcode 的节点，再用 `switch` 处理常量、地址等特殊情况；其余节点进入 TableGen 生成的匹配程序。

这里的实现模型是**虚拟机字节码**，不是有限状态机。`DAGISelMatcherEmitter.cpp` 把 pattern 编译成目标相关的 `static const uint8_t MatcherTable[]`，生成的 `SelectCode` 再调用 `SelectionDAGISel::SelectCodeCommon`。后者逐字节解释 `OPC_Scope`、`OPC_CheckOpcode`、`OPC_CheckType`、`OPC_EmitNode` 等内建操作码，并维护当前 DAG 节点、已记录操作数和失败回溯位置。

```
TableGen pattern
  -> DAGISel matcher 优化
  -> MatcherTable 字节码（uint8_t 数组）
  -> SelectCodeCommon 解释执行
  -> 匹配成功后创建目标 MachineSDNode
```

这里的“字节码”不是 LLVM bitcode，也不是运行时从文件加载的程序；它是编译进目标后端的紧凑匹配表。表中可以包含按 opcode 或类型分派的 switch 操作，但整体执行机制仍是字节码解释器。

因此普通寄存器 `add` 的关键证据是 `.td` 中的 `PatGprGpr<add, ADD>`，而不是手写 `switch` 中某个 `case ISD::ADD`。这是一条很实用的排查规则：

```
测试出现目标指令
  -> 先搜具体 opcode 的 def
  -> 再搜 Pat / multiclass / pattern fragment
  -> 最后检查手写 Select 和 custom lowering
```

你可以用调试选项观察 DAG，但选项会随 LLVM 构建方式和版本变化。若当前构建支持，可尝试：

```
llc -mtriple=riscv64 -O0 -debug-only=isel /tmp/add.ll -o /dev/null
```

Release 构建通常没有这类调试输出。不要把调试选项不可用误判为源码路径不成立。

第四步：MachineInstr 到 MCInst

完成选择后，指令以 `MachineInstr` 存在。后续 pass 会分配物理寄存器、处理 copy、调度和展开 pseudo。本例最终的 `ADD` 进入 `RISCVAsmPrinter::emitInstruction`。

`emitInstruction` 会先验证 feature predicate，处理自动生成的 pseudo lowering 和特殊 pseudo，再把普通指令 lower 成 `MCInst` 并交给 streamer。这里有两条输出路径：

```
MCInst -> asm streamer -> RISCVInstPrinter -> "add a0, a0, a1"
MCInst -> object streamer -> RISCVMCCodeEmitter -> 4 个编码字节
```

`RISCVMCCodeEmitter::encodeInstruction` 查询 `MCInstrDesc`，处理需要展开的 pseudo，然后调用 TableGen 生成的二进制编码逻辑并写出字节。重定位尚不能确定的字段会形成 `MCFixup`，之后由 object writer 处理。

用 MIR 分隔问题

当你无法判断问题发生在指令选择前还是后，可以让 `llc` 停在某个 pass 或使用现有 `.mir` 测试。MIR 把 `MachineFunction` 和 `MachineInstr` 暴露出来，适合研究寄存器分配、frame index 和机器级 pass；LLVM IR 测试更适合验证整条 CodeGen 路径。

实用判断：如果错误的 opcode 已经出现在 MIR 中，继续查 selector 或 lowering；如果 opcode 正确但物理寄存器、栈偏移或最终编码错误，继续查后端较晚阶段。

案例一练习

1. 把第二个操作数改成常数 `1`，预测 `add` 还是 `addi`，再到 pattern 处解释原因。
2. 把类型改为 `i32`，比较 RV64 的输出是否为 `addw`、`add` 或别的形式。注意 IR 的 `i32` 结果是否随后被扩展会影响选择。
3. 为函数加入 `target-features="+c"`，观察最终汇编是否有压缩机会。区分初始 `ADD` 选择和较晚的压缩输出。

阅读记录至少写下：测试路径、输入 IR、目标 triple/features、关键 DAG 操作、选中的 opcode、相关 TableGen pattern、最终输出，以及你尚未解释的差异。

案例二：函数调用与栈帧

先分清四层责任

函数调用不是一个文件完成的。RISC-V psABI 给出外部规则，LLVM 通用 `CCState` 表示分配过程，RISC-V calling-convention 代码决定每个值的位置，frame lowering 再为当前函数组织栈帧。

层	回答的问题	入口
ABI 规范	调用者与被调用者必须共同遵守什么	RISC-V psABI
参数分配	某个参数放寄存器还是栈，怎样扩展	`RISCVCallingConv.cpp/.td`
DAG lowering	怎样产生 copy、load/store、call、return 节点	`RISCVISelLowering.cpp`
栈帧实现	保存哪些寄存器、分配多大栈、怎样引用对象	`RISCVFrameLowering.cpp`、`RISCVRegisterInfo.cpp`

当前源码中的通用 RISC-V ABI 分配有大量手写逻辑在 `RISCVCallingConv.cpp`；`RISCVCallingConv.td` 仍描述 GHC、FastCC 等规则并生成分配函数。阅读时要同时检查 `.cpp` 和 `.td`，不能假定所有规则都在 TableGen 文件中。

用一个带调用的函数提问

考虑下面的 IR：

```
declare i64 @callee(i64, i64)

define i64 @caller(i64 %x, i64 %y) {
  %sum = add i64 %x, %y
  %r = call i64 @callee(i64 %sum, i64 %x)
  ret i64 %r
}
```

对 RV64 的基础整数 ABI，前两个整数参数通常进入 `a0`、`a1`，返回值通常在 `a0`。但不要仅凭汇编记忆推导 LLVM 实现。按以下三条路径分别追踪：

```
函数入口参数 -> LowerFormalArguments
调用点参数与 call -> LowerCall
函数返回值 -> LowerReturn
```

LowerFormalArguments：站在被调用者入口看

在 `RISCVTargetLowering::LowerFormalArguments` 中，先建立 `CCState`，然后调用 `AnalyzeFormalArguments`，传入 `CC_RISCV`。结果保存在一组 `CCValAssign` 中。每个 `CCValAssign` 记录原值类型、位置类型、位置是寄存器还是内存，以及需要 full、sign-extend、zero-extend、bit-convert 或 indirect 等处理。

接下来 lowering 根据分配结果：

- 对寄存器参数建立 live-in 和 `CopyFromReg`。
- 对栈上传入的参数建立 fixed stack object，再生成 load。
- 对间接传递的值，先取得地址再读取对象。
- 对可变参数，记录未使用参数寄存器和栈区域，以支持 `va_start` 等操作。

观察 `CCValAssign` 比只盯着 `a0` 更重要，因为它解释了“IR 值”和“ABI 位置”之间的类型转换。

LowerCall：站在调用者看

`RISCVTargetLowering::LowerCall` 同样先分析 outgoing arguments。它需要完成的工作包括：

1. 计算栈上传参区域并建立 `CALLSEQ_START`。
2. 把栈参数存入约定偏移。
3. 用 `CopyToReg` 把寄存器参数送入 `a0` 到 `a7` 或浮点参数寄存器。
4. 构造目标相关 call 节点，并声明 call-preserved mask。
5. 建立 `CALLSEQ_END`。
6. 从返回值寄存器复制结果。

这里的 register mask 告诉寄存器分配器：一次调用可能破坏哪些寄存器。callee-saved 与 caller-saved 的区别因此会直接影响 live value 是否需要保存。

直接调用在较晚阶段可能仍以 `PseudoCALL` 表示，因为符号距离和链接时 relaxation 尚未确定。`RISCVMCCodeEmitter::encodeInstruction` 对 call pseudo 有专门展开逻辑，通常涉及 `auipc` 与 `jalr` 以及 fixup。它再次说明 pseudo 可以一直存活到 MC 层附近。

LowerReturn：把结果放回 ABI 位置

`RISCVTargetLowering::LowerReturn` 分析返回值位置，把结果 `CopyToReg` 到约定寄存器，并产生 RISC-V return 节点。对于普通整数返回值常见位置是 `a0`、`a1`；更复杂的聚合、浮点或 scalable vector 需要按 ABI 和当前 feature 继续分析。

`ret` 汇编通常是 `jalr x0, 0(ra)` 的别名。看到文本 `ret` 时，要知道汇编别名、MachineInstr opcode 和真实编码不一定使用同一个名字。

emitPrologue 和 emitEpilogue：建立与撤销栈帧

`RISCVFrameLowering::emitPrologue` 在寄存器分配和 frame layout 已获得足够信息后插入序言。它会根据栈大小、栈对齐、callee-saved 保存、frame pointer、动态栈调整和目标扩展选择具体序列。`emitEpilogue` 在返回前执行相反方向的恢复。

概念上的常见过程是：

```
入口: sp -= stack_size
      保存 ra / s0... 或使用相应保存机制
      必要时建立 fp

出口: 恢复 callee-saved 寄存器
      sp += stack_size
      return
```

真实顺序会受 shrink wrapping、大栈调整、可变大小对象、栈探测和扩展指令影响。源码阅读时先跟随控制流中的“是否需要 frame pointer”“栈大小是否可由立即数表达”“是否有 callee-saved info”三个判断。

eliminateFrameIndex：把抽象栈对象变成地址

较早的 MachineInstr 可以用 frame index 表示某个栈对象，而不立即决定它相对 `sp` 或 `fp` 的最终偏移。完成 frame layout 后，`RISCVRegisterInfo::eliminateFrameIndex`：

1. 取得指令中的 frame index 操作数。
2. 根据对象位置、栈大小和 frame register 计算最终偏移。

3 若偏移能装入指令立即数字段，直接改写 base register 与 immediate。

4 若偏移过大，先用额外指令把偏移物化到寄存器，再形成地址。

所以“栈偏移太大时为什么多出几条指令”通常要查 frame index 消除，而不是查 IR lowering。

叶子函数检查清单

叶子函数没有普通函数调用，但不等于一定没有栈帧。阅读输出时依次检查：

- 是否有局部栈对象或 spill。
- 是否使用 callee-saved 寄存器。
- 是否需要 frame pointer。
- 是否有动态 alloca、特殊对齐或栈保护。
- 是否因调试、插桩或目标选项增加序言工作。
- `ra` 是否真的需要保存。

若一个只做 `add` 的叶子函数直接 `ret`，这是“没有需要保存的状态”的结果，不是后端专门写死了“叶子函数无栈帧”。

案例二练习

1 创建 10 个 `i64` 参数的函数。预测前 8 个整数参数与其余参数的位置，再用 `LowerFormalArguments` 的分配结果解释。

2 比较有调用和无调用函数中 `ra` 的处理。

3 添加一个较大的局部数组，观察栈调整和 frame index 地址生成。

4 用 `-mabi=lp64` 与 `-mabi=lp64d` 编译含浮点参数的 C 函数，先确认 `-march` 与 ABI 相容，再追踪参数分配差异。

完成本案例后，你应能把一个调用相关问题先分类为参数分配、call lowering、寄存器保存、frame layout 或 MC/linker fixup，而不是只搜索 `call` 字符串。

案例三：RVV 与 vsetvli

先分清中端向量化与后端代码生成

Loop Vectorizer 属于 LLVM 中端：它分析循环是否合法且有利可图，并把标量 LLVM IR 改写成向量 IR。RISC-V 后端接收向量 IR、VP intrinsic 或 RISC-V intrinsic，选择 RVV pseudo，分配向量寄存器，再插入需要的 `vsetvli`。

必须明确分层：自动向量化属于中端；`vsetvli` 插入属于 RISC-V 后端。后端不会替 Loop Vectorizer 证明一个循环能否向量化，Loop Vectorizer 也不会直接安排某个基本块前应放哪条 `vsetvli`。

```
标量循环 IR
  |
  | Loop Vectorizer（中端）
  v
向量 IR / scalable vector / VP intrinsic
  |
  | RISC-V legalization + instruction selection（后端）
  v
带 AVL、SEW、policy 操作数的 RVV pseudo
  |
  | RISCVInsertVSETVLI（后端 MachineFunction pass）
  v
```

```
显式 vsetvli + RVV 指令
```

为什么 scalable vector 很关键

RVV 的物理向量长度 VLEN 由具体实现决定。LLVM scalable vector 类型写作 `<vscale x N x Ty>`，表示元素数是运行时 `vscale` 与最小元素数 `N` 的乘积。它避免编译器假定固定 VLEN。

例如 `<vscale x 4 x i32>` 不是“永远 4 个 i32”，而是每个 `vscale` 单位至少包含 4 个 i32 元素。后端需要结合 SEW 与 LMUL 表达所需寄存器组和 VLMAX。

先掌握四个配置概念

概念	含义	在后端中为什么重要
AVL	Application Vector Length，本次希望处理的元素数	决定实际 `vl`，可能来自寄存器、立即数或 VLMAX 请求
SEW	Selected Element Width	决定当前向量元素宽度，例如 e8、e32、e64
LMUL	向量寄存器组倍率	影响寄存器组大小和 VLMAX
policy	tail/mask 的 agnostic 或 undisturbed 策略	决定非活动元素是否必须保留旧值

硬件用 `vl` 和 `vtype` 状态控制后续向量指令。`vsetvli` 根据 AVL 与 vtype 设置这些状态。因此两条要求不同 SEW、LMUL 或 policy 的向量指令之间，可能必须插入新的配置指令。

从测试反查 RVV pseudo

先选择小测试，而不是直接进入 7000 行的 `.td` 文件：

- IR 到 RVV 指令：`llvm/test/CodeGen/RISCV/rvv/vadd-vp-mask.ll`
- 专门检查配置插入：`llvm/test/CodeGen/RISCV/rvv/vsetvli-insert.mir`
- 跨基本块传播：`llvm/test/CodeGen/RISCV/rvv/vsetvli-insert-crossbb.ll`
- 回归场景：`llvm/test/CodeGen/RISCV/rvv/vsetvli-regression.ll`

在 `RISCVInstrInfoV.td` 中，`defm VADD_V : VALU_IV_V_X_I<"vadd", ...>` 描述真实 RVV 指令族。`RISCVInstrInfoVPseudos.td` 中的 `defm PseudoVADD : VPseudoVALU_VV_VX_VI` 则展开出不同 LMUL、masked/unmasked 和 operand form 的 pseudo，并有 pattern 把 intrinsic 或 DAG 操作映射过去。

为何需要如此多 pseudo？因为在插入实际 `vsetvli` 前，每条向量操作仍要携带配置需求。pseudo 的显式操作数或描述信息可以包含 AVL、SEW 和 policy；真实 `vadd.vv` 依赖隐式的硬件 `vl/vtype` 状态，单靠最终 opcode 无法保留所有分析信息。

RISCVInsertVSETVLI 的三阶段数据流

入口是 `RISCVInsertVSETVLI::runOnMachineFunction`。它先通过 `hasVInstructions()` 跳过不支持向量的函数，并为每个基本块维护 `BlockData`：

```
BlockData.Exit = 当前块出口处已知的 VL/VTYPE 状态
BlockData.Pred = 所有前驱汇合后能保证的入口状态
```

源码注释给出三阶段算法：

```
Phase 1: 逐块扫描，计算每个块内部怎样改变 VL/VTYPE
    |
    v
Phase 2: 工作队列数据流，计算前驱汇合状态和各块出口状态
```

```
    |
    +-> doPRE: 对配置转换做部分冗余消除
    v
Phase 3: 按需要在块内插入 vsetvli
    |
    v
局部 coalesce / demanded-fields 优化与后续清理
```

Phase 1 的 `computeVLVTYPEChanges` 概括一个块的 transfer effect。Phase 2 的 `computeIncomingVLVTYPE` 反复传播，直到跨基本块状态稳定。Phase 3 的 `emitVSETVLIs` 检查前驱已经提供的状态能否满足第一条向量指令，避免无条件在每个块开头插入配置。

`needVSETVLI` 比较当前状态和指令需求。比较不是简单要求所有字段完全相等，因为某条操作可能只 demand 部分字段。`insertVSETVLI` 再根据 AVL 和前一状态选择合适 pseudo；例如 AVL 相同且 VLMAX 相容时，可能使用 `x0, x0` 形式。

配置状态怎样穿过控制流

考虑菱形 CFG：

```
          entry: e32,m1
           /          \
 left: e32,m1      right: e16,m1
           \          /
              merge
             需要 e32,m1
```

若两个前驱出口状态不同，merge 块入口不能假定 `e32,m1`，因此第一条需要该配置的向量指令前要插入 `vsetvli`。如果两个前驱都保证同一兼容状态，merge 块可复用它。PRE 还可能把转换放到部分前驱，使公共路径少执行配置。

用集合的眼光理解汇合更稳妥：`Pred` 只保留所有实际前驱共同保证、且足以满足后续需求的信息。任何路径上可能被改变的字段都不能凭空当作已知。

tail 与 mask policy 为什么影响优化

tail agnostic/mask agnostic 允许硬件不保留相应非活动元素；undisturbed 则要求保留原值。后端在选择 masked pseudo 和 passthru operand 时必须反映语义，`RISCVInsertVSETVLI` 也要让 vtype policy 满足指令需求。

如果你只比较 SEW 和 LMUL，可能误以为两条向量指令能共用配置。阅读测试检查行时要同时看 `ta/tu` 与 `ma/mu`。某些变换能把更严格的状态用于需求较弱的操作，具体兼容条件以 `DemandedFields` 和 `needVSETVLI` 为准。

AVL、SEW、LMUL 的一个推导练习

对一个 `<vscale x 4 x i32>` 加法：

1. 元素类型给出 SEW=32 的核心需求。
2. LLVM 向量类型和目标 lowering 共同决定相应 LMUL/寄存器类。
3. VP intrinsic 的 explicit vector length 或普通 scalable 操作的 VLMAX 语义决定 AVL 形态。
4. policy 决定是否需要 passthru 以及 vtype 中的 tail/mask 位。
5. selector 产生带这些需求的 `PseudoVADD...`。
6. `RISCVInsertVSETVLI` 比较当前状态，必要时插入配置。
7. 较晚 pseudo expansion 映射为真实 `VADD_VV` 等 opcode。

这里不要把“LLVM 类型中的最小元素数”直接等同于运行时 VL。AVL 可能小于、等于或用于请求 VLMAX，硬件最后产生的 `vl` 还受 VLEN、SEW 和 LMUL 约束。

案例三练习

1. 打开 `vsetvli-insert.mir`，选一个只有一个基本块的测试。先遮住 CHECK 行，按 pseudo 的 AVL/SEW/policy 预测配置位置。
2. 在相邻两条向量 pseudo 之间改变 SEW，预测是否必须出现第二条 `vsetvli`。
3. 再找一个配置相同的相邻操作，解释 pass 为什么能复用状态。
4. 阅读 `vsetvli-insert-crossbb.ll` 的一个 CFG，画出每个块的 `Pred` 与 `Exit`，再与 CHECK 结果比较。
5. 从 `PseudoVADD` 出发，反查 `RISCVInstrInfoVPseudos.td` 的生成类、真实 `VADD_V` 定义和一个对应测试，记录 masked 与 unmasked 版本的操作数差异。

完成本案例后，你应该能判断一个问题发生在中端向量化、后端向量合法化/选择，还是 `RISCVInsertVSETVLI` 的机器级状态传播。

形成自己的源码阅读方法

从测试反查实现的七步法

1. **固定现象。** 找到最小 CodeGen 测试，抄下 `RUN`、triple、CPU、features、ABI 和优化级别。
2. **写出数据形态。** 标注当前问题位于 LLVM IR、SelectionDAG、MachineInstr、MCInst 还是对象文件。
3. **从输出 opcode 搜定义。** 在 `.td` 中找真实指令、pseudo、pattern 和 predicate。
4. **从输入操作搜 lowering。** 检查 `RISCVISelLowering` 的合法化 action、custom lowering 和 combine。
5. **检查 selector 边界。** 普通 pattern 看生成规则，特殊节点看 `RISCVISelDAGToDAG::Select`。
6. **追踪较晚机器 pass。** 栈、寄存器、pseudo、RVV 状态和压缩等问题在 MachineInstr 阶段继续查。
7. **回到测试做单变量实验。** 一次只改类型、常数、feature、ABI 或 CFG 中的一项，并记录预测与结果。

这套顺序的核心是始终知道当前表示。许多无效搜索都来自把 IR 名称、DAG opcode、MachineInstr opcode、汇编别名混为一谈。

源码阅读记录模板

```
问题：
源码基准提交：043acd707
最小测试路径：
RUN 行与关键选项：
输入 IR / MIR：
预期输出：

当前表示：IR / SelectionDAG / MachineInstr / MCInst / object
关键目标特性：
关键 ABI：

定位链：
1. 测试 ->
2. lowering/legalization ->
3. TableGen pattern/pseudo ->
4. selector ->
5. machine pass ->
6. AsmPrinter/MC ->

已验证证据：
仍未确认：
下一次只改变的变量：
```

推荐阅读顺序

第一轮只读测试和几个稳定入口：

1. `llvm/test/CodeGen/RISCV/alu64.ll`
2. `RISCVCodeGenPassBuilder.cpp`
3. `RISCVISelLowering.h` 的类接口
4. `RISCVInstrInfo.td` 中 ADD 定义与简单 pattern
5. `RISCVISelDAGToDAG.cpp` 的 `Select`
6. `RISCVAsmPrinter.cpp` 的 `emitInstruction`
7. `RISCVMCCodeEmitter.cpp` 的 `encodeInstruction`

第二轮加入 ABI 和栈帧：`RISCVCallingConv.cpp/.td`、三个 call lowering 函数、`emitPrologue`、`emitEpilogue`、`eliminateFrameIndex`。

第三轮再进入 RVV：先读一个 `.ll` 测试和一个 `.mir` 测试，然后读 `RISCVInstrInfoV.td`、`RISCVInstrInfoVPseudos.td`，最后读 `RISCVInsertVSETVLI::runOnMachineFunction` 及它调用的阶段函数。

常用观察工具

工具或选项	适合观察	注意点
`rg`	稳定符号、测试和 TableGen 记录	优先搜类/函数/opcode，行号仅辅助
`llc -o -`	最终汇编	先记录 `llc --version`
`llc -stop-after=...`	某个 pass 后的 MIR	pass 名会随版本变化
`-print-before/after`	pass 前后表示变化	输出量很大，应限制函数或 pass
`llvm-mc`	汇编到编码、feature 诊断	属于 MC 层，不经过完整 CodeGen
`llvm-objdump -d`	目标文件反汇编	汇编别名可能掩盖真实 opcode 名称
`FileCheck`	稳定表达测试期望	检查关键性质，避免无意义过度约束

核心术语表

术语	本教程中的含义
Legalization	把目标不支持的类型或操作改写为可处理形式
Lowering	从较抽象操作下降到目标相关表示，可能发生在多个阶段
SelectionDAG	本教程主线使用的 DAG 指令选择框架
MachineInstr	LLVM 机器代码层的指令表示，可含虚拟寄存器和 pseudo
MCInst	MC 层的轻量指令表示，用于文本输出或编码
TableGen	从声明式记录生成指令信息、matcher、编码器等 C++ 表的工具
MatcherTable	TableGen 为 SelectionDAG 指令选择生成的目标相关匹配字节码
Pseudo instruction	暂时表达复杂语义、稍后展开或 lower 的机器指令
Calling convention	参数、返回值和寄存器保存等函数调用规则
Frame index	栈对象的抽象编号，frame layout 后改写成实际地址
Scalable vector	元素数随运行时 `vscale` 成比例变化的 LLVM 向量类型
AVL	应用请求的向量长度
SEW	RVV 当前选定元素位宽
LMUL	RVV 向量寄存器组倍率
VTYPE	包含 SEW、LMUL 和 policy 等配置的硬件状态
VLMAX	给定 VLEN、SEW 和 LMUL 时可处理的最大元素数

最后一次自测

在不看正文的情况下，尝试回答：

1. 为什么 RV64 `add i64` 可匹配一条 `ADD`，RV32 却不能？
2. `.td` 中的指令定义、pattern 和 `RISCVISelDAGToDAG::Select` 各承担什么？
3. `LowerCall` 与 `emitPrologue` 为什么不能合并理解成同一步？
4. frame index 为什么要延迟到 frame layout 后再消除？
5. Loop Vectorizer 和 `RISCVInsertVSETVLI` 分别处于哪一层？
6. 为什么 RVV pseudo 需要携带 AVL、SEW、LMUL 或 policy 信息？
7. 一个 merge 块能否复用前驱配置，取决于什么数据流事实？

如果你能从测试路径出发，为每题指出至少一个具体源码入口，就已经建立了可继续扩展的 RISC-V LLVM 后端阅读方法。

参考源码路径

以下路径均相对于 `/root/MIC-for-learning/llvm-project`：

- `llvm/test/CodeGen/RISCV/alu64.ll`
- `llvm/test/CodeGen/RISCV/rvv/vadd-vp-mask.ll`

- llvm/test/CodeGen/RISCV/rvv/vsetvli-insert.mir
- llvm/test/CodeGen/RISCV/rvv/vsetvli-insert-crossbb.ll
- llvm/lib/Target/RISCV/RISCVTargetMachine.cpp
- llvm/lib/Target/RISCV/RISCVCodeGenPassBuilder.cpp
- llvm/lib/Target/RISCV/RISCVSubtarget.cpp
- llvm/lib/Target/RISCV/RISCVISelLowering.cpp
- llvm/lib/Target/RISCV/RISCVISelDAGToDAG.cpp
- llvm/lib/Target/RISCV/RISCVInstrInfo.td
- llvm/lib/Target/RISCV/RISCVCallingConv.cpp
- llvm/lib/Target/RISCV/RISCVCallingConv.td
- llvm/lib/Target/RISCV/RISCVFrameLowering.cpp
- llvm/lib/Target/RISCV/RISCVRegisterInfo.cpp
- llvm/lib/Target/RISCV/RISCVAsmPrinter.cpp
- llvm/lib/Target/RISCV/MCTargetDesc/RISCVMCCodeEmitter.cpp
- llvm/lib/Target/RISCV/RISCVInstrInfoV.td
- llvm/lib/Target/RISCV/RISCVInstrInfoVPseudos.td
- llvm/lib/Target/RISCV/RISCVInsertVSETVLI.cpp
- llvm/docs/RISCVUsage.md
- llvm/docs/RISCV/RISCVVectorExtension.md